Fully Sparse Fusion for 3D Object Detection

Yingyan Li, Lue Fan, Yang Liu, Zehao Huang, Yuntao Chen, Naiyan Wang, and Zhaoxiang Zhang

***Abstract*—Currently prevalent multi-modal 3D detection methods rely on dense detectors that usually use dense Bird's-Eye-View (BEV) feature maps. However, the cost of such BEV feature maps is quadratic to the detection range, making it not scalable for long-range detection. Recently, LiDAR-only fully sparse architecture has been gaining attention for its high efficiency in long-range perception. In this paper, we study how to develop a multi-modal fully sparse detector. Specifically, our proposed detector integrates the well-studied 2D instance segmentation into the LiDAR side, which is parallel to the 3D instance segmentation part in the LiDAR-only baseline. The proposed instance-based fusion framework maintains full sparsity while overcoming the constraints associated with the LiDAR-only fully sparse detector. Our framework showcases state-of-the-art performance on the widely used nuScenes dataset, Waymo Open Dataset, and the long-range Argoverse 2 dataset. Notably, the inference speed of our proposed method under the long-range perception setting is 2.7× faster than that of other state-of-the-art multimodal 3D detection methods.**

***Index Terms*—3D object detection, multi-sensor fusion, fully sparse architecture, autonomous driving, long-range perception.**

I. Introduction

3D OBJECT detection serves as a crucial component in autonomous driving systems. Currently, LiDARs and cameras are the two main sensors used for detection. Combining these two sensors [1], [2], [3] is popular in current perception system, which leverages precise depth of LiDARs and rich semantics of cameras. Currently, the predominant multi-modal methods [1], [2], [3] rely on dense detectors. Dense detectors imply the detectors construct dense Bird's-Eye-View (BEV) feature maps for prediction as Fig. 1 shows. The size of these BEV feature maps increases quadratically with perception range, which leads to unaffordable costs in long-range detection.

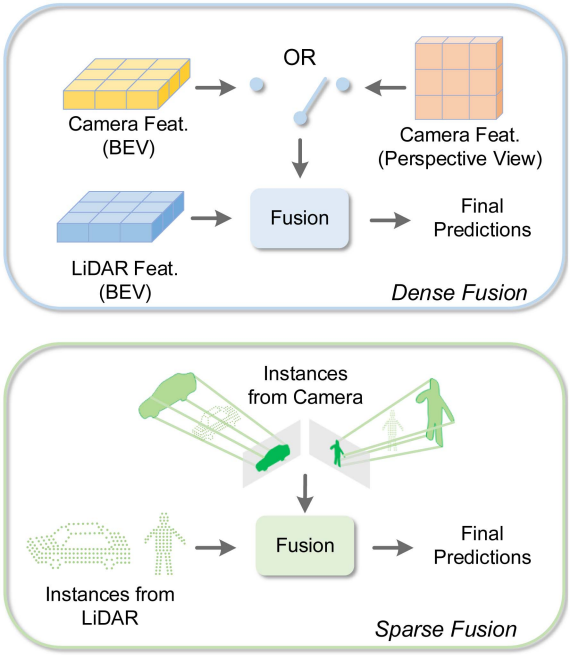

Fig. 1. Comparison between dense fusion and sparse fusion. Dense fusion methods rely on dense BEV feature maps. In contrast, our sparse fusion framework fuses two modalities at the instance level, requiring no dense feature maps.

To solve this problem, we aim to design a multi-modal detector without introducing any dense BEV features. Recently, several LiDAR-only fully sparse detectors [4], [5], [6], [7] have emerged, where fully sparse implies that the whole architecture does not include any BEV feature map. As a representative, Fully Sparse Detector (FSD) [4] demonstrates impressive efficiency and efficacy. However, given FSD, how to build a fully sparse multimodal 3D detector that guarantees efficiency and effectiveness remains an unknown issue. On the one hand, we need to ensure that the entire framework remains fully sparse, without introducing BEV features. On the other hand, we aim to effectively address the shortcomings of FSD. FSD relies on clustering-based 3D instance segmentation, which makes it easy to ignore the foreground objects with few points, and hard

Manuscript received 26 November 2023; revised 13 March 2024; accepted 7 April 2024. Date of publication 22 April 2024; date of current version 3 October 2024. This work was supported in part by the National Key R&D Program of China under Grant 2022ZD0160102, in part by the National Natural Science Foundation of China under Grant U21B2042 and Grant 62072457, and in part by the 2035 Innovation Program of CAS. Recommended for acceptance by G. Huang. *(Yingyan Li and Lue Fan contributed equally to this work.) (Corresponding author: Zhaoxiang Zhang.)*

Yingyan Li and Lue Fan are with the Center for Research on Intelligent Perception and Computing (CRIPAC), State Key Laboratory of Multimodal Artificial Intelligence Systems (MAIS), Institute of Automation, Chinese Academy of Sciences (CASIA), Beijing 100190, China, and also with the School of Future Technology, University of Chinese Academy of Sciences (UCAS), Beijing 100049, China (e-mail: liyingyan2021@ia.ac.cn; fanlue2019@ia.ac.cn).

Yang Liu is with the Center for Research on Intelligent Perception and Computing (CRIPAC), State Key Laboratory of Multimodal Artificial Intelligence Systems (MAIS), Institute of Automation, Chinese Academy of Sciences (CASIA), Beijing 100190, China (e-mail: liuyang2022@ia.ac.cn).

Zehao Huang and Naiyan Wang are with TuSimple, Beijing 100020, China (e-mail: zehaohuang18@gmail.com; winsty@gmail.com).

Yuntao Chen is with the Centre for Artificial Intelligence and Robotics, Hong Kong Institute of Science and Innovation, Chinese Academy of Sciences (HKISI CAS), Hong Kong, China (e-mail: chenyuntao08@gmail.com).

Zhaoxiang Zhang is with the Center for Research on Intelligent Perception and Computing (CRIPAC), State Key Laboratory of Multimodal Artificial Intelligence Systems (MAIS), Institute of Automation, Chinese Academy of Sciences (CASIA), Beijing 100190, China, also with the School of Future Technology, University of Chinese Academy of Sciences (UCAS), Beijing 100049, China, and also with the Centre for Artificial Intelligence and Robotics, Hong Kong Institute of Science and Innovation, Chinese Academy of Sciences (HKISI CAS), Hong Kong, China (e-mail: zhaoxiang.zhang@ia.ac.cn).

Code is released at https://github.com/BraveGroup/FullySparseFusion.

Digital Object Identifier 10.1109/TPAMI.2024.3392303

0162-8828 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

to differentiate closely adjacent objects, leading to low object recall.

Therefore, we propose Fully Sparse Fusion (FSF), a detector concentrating on addressing the above two challenges from the perspective of instance-based fusion. First, we use the well-studied 2D instance segmentation model to yield 2D masks, which possess enhanced object recall due to the rich semantics and high resolution of images. Next, we transform each 2D instance mask into a 3D frustum, according to the intrinsic and extrinsic parameters of cameras. The points contained within this frustum constitute a 3D instance corresponding to the mask. These 3D instances from image modality are complementary to those 3D instances generated from LiDAR-based instance segmentation in FSD. Finally, a bi-modal instance-based prediction module is leveraged to fuse these two kinds of instances and make final predictions. This module is featured with a couple of sub-modules including instance feature extraction/interaction, instance shape alignment, and a prediction head. Furthermore, due to the different shapes and spatial distributions of the bi-modal instances, conventional label assignment is insufficient. To this end, we propose a two-stage assignment to assign labels to the instances from two modalities.

The proposed framework offers multiple benefits. First, it is totally based on instances without introducing any dense BEV features, guaranteeing full sparsity. Additionally, it gets the best of two worlds by seamlessly integrating well-studied 2D instance segmentation and 3D instance segmentation, mitigating the low recall of the LiDAR side and facilitating highly accurate object localization. Due to incorporating instance-based information, FSF is superior to the semantic-only fusion approaches such as PointPainting [8] and PointAugmenting [9].

We summarize our contributions as follows:

- We propose Fully Sparse Fusion (FSF), a novel fully sparse multi-modality perception framework, which leverages instance-based fusion without any dense BEV feature maps.
- FSF seamlessly integrates 2D instance segmentation and 3D instance segmentation by bi-modal instance generation and bi-modal instance-based prediction, making the best of two worlds.
- FSF achieves state-of-the-art performance in the nuScenes dataset [10], Waymo Open Dataset [11], and Argoverse 2 dataset [12]. Particularly, for the long-range detection in the Argoverse 2 dataset, FSF is 2.7 $\times$ faster than the previous state-of-the-art multi-modal detector.

II. Related Work

A. Camera-Based 3D Detection

In the early years of camera-based 3D detection, the focus is mainly on monocular 3D detection [13], [14], [15], [16], [17], [18], [19]. However, monocular 3D detection ignores the relationships between cameras. To this end, more and more researchers are engaged in multi-view 3D detection. Multi-view 3D detection [20], [21], [22] contains two main types of approaches: BEV-based and object-query-based. BEV-based methods [23], [24] lift 2D features to 3D [25], [26], [27] or using transformer-like structures [22], [24], [28] to construct BEV feature map for prediction. On the contrary, object-query-based methods [29], [30], [31] predict 3D bounding boxes from the features of object queries.

B. LiDAR-Based 3D Detection

Although camera-based 3D detection methods gain great improvements in the past few years, their performance is still inferior to LiDAR-based 3D detection methods since LiDAR points have accurate depths. LiDAR-based methods [32], [33] commonly transform irregular LiDAR point clouds into regular spaces, like 2D bird's-eye view or 3D voxels, to facilitate feature extraction. PointPillars [32], VoxelNet [33] and 3DFCN [34] employ dense 2D/3D convolution to extract features. However, utilizing dense convolution introduces heavy computational costs. To address this issue, SECOND [35] applies sparse convolution to 3D detection and inspires an increasing number of methods [36], [37], [38], [39] adopting sparse convolution for feature extraction. However, even with sparse convolution, most methods [36], [40] still require a dense feature map to alleviate the so-called "center feature missing" issue [4]. Researchers have been dedicated to developing fully sparse methods to better utilize the sparsity nature of points. The pioneering works in this domain are PointNet-like methods [41], [42], [43], [44], [45]. Also, range view [46], [47] requires no BEV feature map, though their performance is inferior. Recently, FSD [4] has been proposed and demonstrated faster speed and higher accuracy than previous methods. Our work is based on FSD.

C. Multi-Modal 3D Detection

The multi-modal 3D detection approaches can be classified into two types, namely dense and sparse, depending on whether it relies on the dense BEV feature map.

Dense Fusion: As the representation of dense frameworks, BEVFusion [1], [2] fuses the BEV feature maps generated from camera and LiDAR modalities. Another line of dense fusion frameworks [3], [48], [49], [50], [51], [52] relies on the BEV feature map to generate proposals. There is a concurrent work named SparseFusion [53]. However, SparseFusion relies on BEV feature maps to generate instances, which means it is also a dense fusion framework.

Sparse Fusion: The sparse approach, on the other hand, does not rely on dense feature maps. The most popular approaches are the point-level methods [8], [54], [55], which project LiDAR points onto the image plane to obtain image semantic cues. PointPainting [8] paints semantic scores, while PointAugmenting [9] paints image features. Also, there are some instance-level fusion [56], [57], [58] methods. MVP [59] uses instance information to produce virtual points to enrich point clouds. FrustumPointNet [60] and FrustumConvnet [61] utilize frustums to generate proposals. These methods work without relying on dense feature maps.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

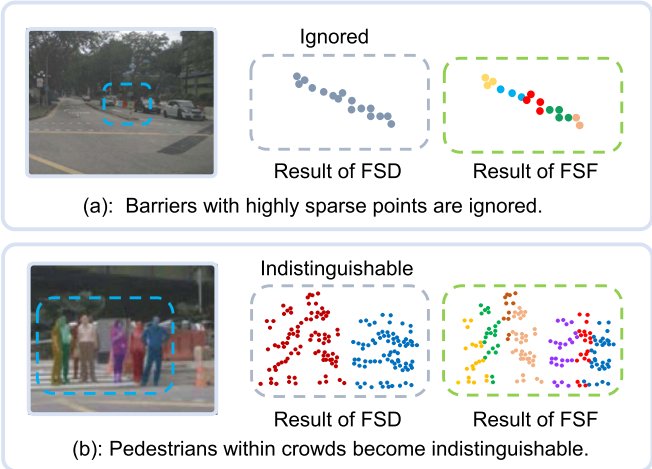

Fig. 2. (a) 3D instance segmentation is prone to ignore objects whose points are few. (b) It is hard for 3D segmentation to separate the overlapped objects in a crowded scene. On the contrary, handling these cases via 2D instance segmentation is easier.

III. Preliminary: Fully Sparse 3D Detector

We first briefly introduce the emerging fully sparse 3D object detector FSD [4], which is adopted as our LiDAR-only baseline. FSD is divided into two modules: Points-to-Instances and Instances-to-Boxes.

A. Points-to-Instances

The Points-to-Instances module performs 3D instance segmentation in two steps: foreground point extraction and foreground point grouping. First, FSD classifies each point to gain the set of foreground points. Then, the foreground points are aggregated into groups according to their spatial proximity. A group of foreground points is denoted as a 3D instance.

B. Instances-to-Boxes

The instances-to-boxes module is divided into two parts: instance feature extraction and instance-based prediction. First, given the instances, a PointNet-like instance feature extractor, named Sparse Instance Recognition (SIR) module is utilized for feature extraction. Next, a simple MLP head predicts 3D bounding boxes based on extracted instance features.

The structure mentioned above does not involve any dense BEV feature maps. Thus it can be easily extended to long-range scenarios.

C. Limitations of Single-Modal FSD

While FSD showcases impressive capabilities, it still has several limitations. The capabilities of FSD greatly depend on the effectiveness of 3D instance segmentation. However, in scenarios with highly sparse point clouds, the system might miss detecting some objects. Additionally, when multiple objects are closely spaced, the system might mistakenly group them as one, as shown in Fig. 2.

We develop our Fully Sparse Fusion (FSF) framework, which enriches the fully sparse architecture with image information. Incorporating image information provides a feasible solution to these issues.

IV. Methodology

A. Overall Architecture

The overall architecture of FSF is shown in Fig. 3. The framework is divided into two parts: the Bi-modal Instance Generation module in Section IV-B and the Bi-modal Instance-based Prediction module in Section IV-C. The Bi-modal Instance Generation module produces LiDAR instances in Section IV-B1 and camera instances in Section IV-B2. Then, the Bi-modal Instance-based Prediction module aligns the shape of the bi-modal instances and produces final bounding boxes based on these instances.

B. Bi-Modal Instance Generation

The Bi-modal Instance Generation module primarily consists of two parts: LiDAR Instance Generation module in Section IV-B1 and Camera Instance Generation module in Section IV-B2. The LiDAR Instance Generation uses 3D instance segmentation to generate LiDAR instances. Meanwhile, the Camera Instance Generation module uses 2D instance segmentation to create masks and then generates camera instances based on these masks.

1) LiDAR Instance Generation: Given a point cloud, we utilize 3D instance segmentation to generate a set of LiDAR instances following FSD. The detailed algorithm is introduced as follows.

First, we extract foreground points from the point cloud. Specifically, the point cloud undergoes voxelization, converting the point cloud into a set of voxels. A sparse voxel encoder extracts voxel features from these voxels. Given voxel features, we obtain the feature of each point by concatenating the coordinate of this point with the corresponding voxel feature. Using the point features, we classify each point with a simple MLP, producing the foreground score of this point. We retain only the points with classification scores higher than τ as foreground points. The foreground point set is denoted as $\mathbf{F}$.

Next, we group $\mathbf{F}$ into 3D instances. Specifically, each foreground point predicts a proxy point. This proxy point represents the center of the object to which this foreground point belongs. We call these proxy points "voted centers". The set of voted centers is denoted as $\mathbf{C}$. Then, the *Connected Components Labeling (CCL)* algorithm [62] is employed to generate instances by connecting voted centers. Within the algorithm, two voted centers are considered "connected" if their distance falls below a threshold θ. CCL groups the adjacent voted centers and forms m^L groups. The i-th center group is denoted as $\mathbf{C}_i$, where $i = 1, \ldots, m^L$. Given that a voted center corresponds one-to-one with a foreground point, we use $\mathbf{C}_i$ to construct the corresponding foreground point set $\mathbf{F}_i$, where each foreground point in $\mathbf{F}_i$ corresponds its voted center in $\mathbf{C}_i$. The i-th LiDAR instance is $\mathbf{I}_i^L = \mathbf{F}_i$.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

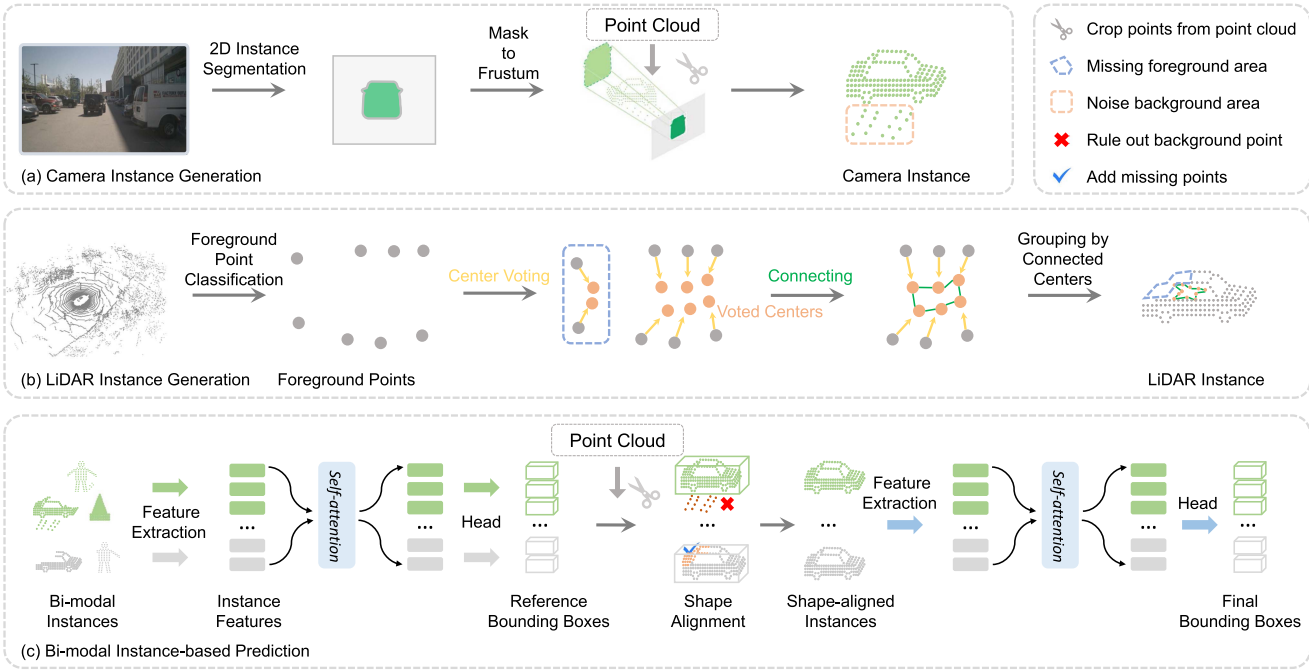

Fig. 3. The overview of our framework. Our framework is mainly divided into two parts: Bi-modal Instance Generation module in Section IV-B and Bi-modal Instance-based Prediction module in Section IV-C. Bi-modal Instance Generation module generates instances from camera and LiDAR modalities. Then, the Bi-modal Instance-based Prediction module aligns the shapes of bi-modal instances and produces the final bounding boxes.

During training, we consider all points that fall within the ground-truth bounding box as the ground-truth foreground points. The centers of the ground-truth bounding boxes serve as the ground-truth voting centers for these foreground points.

2) Camera Instance Generation: Camera Instance Generation module is a module parallel to LiDAR Instance Generation module for camera instance generation. We introduce this module as follows.

First, we project the 3D LiDAR points into a 2D image plane. Given the Intrinsic matrix $\mathbf{K}$ and Extrinsic matrix $[\mathbf{R}|\mathbf{t}]$ of a camera, the point cloud $\mathcal{P}$ are projected onto a 2D image plane to obtain the 2D points $\mathcal{U}$ by

$$\mathcal{U} = \mathbf{K}[\mathbf{R}|\mathbf{t}]\mathcal{P}. \tag{1}$$

We neglect the homogeneous coordinates for simplicity.

Then, given the projected points, we utilize 2D instance segmentation to generate camera instances. Specifically, we produce n instance masks from a single-view image by the well-studied 2D instance segmentation model [63]. We represent the j-th mask as M_j, where $j = 1, \ldots, n$. Subsequently, we collect the 2D points that fall within M_j, defined as $\mathcal{U}_j$. The corresponding 3D points denoted as $\mathcal{P}_j$. As a result, we obtain the j-th camera instance as $\mathbf{I}_j^C = \mathcal{P}_j$. Geometrically, this process is equal to lifting a 2D mask into a 3D frustum space and collecting all the 3D points falling in this frustum.

In multi-view scenes, overlapping of frustums from different cameras occurs. To resolve this, we employ a point clone strategy for the regions where frustums intersect. Specifically, points located in the overlapping regions are cloned and allocated to each intersecting frustum. As a result, each frustum is guaranteed to have all relevant points.

So far, we have obtained LiDAR instances $\mathbf{I}^L = \{\mathbf{I}_1^L, \ldots, \mathbf{I}_{m^L}^L\}$ and camera instances $\mathbf{I}^C = \{\mathbf{I}_1^C, \ldots, \mathbf{I}_{m^C}^C\}$, where m^L, m^C represents the number of LiDAR instances and the camera instances captured from all cameras.

C. Bi-Modal Instance-Based Prediction

We have generated bi-modal instances. However, the generation methods of LiDAR instances and camera instances exhibit several limitations. The generation of LiDAR instances is based on connecting the voted centers. When the distance between voted centers is greater than the threshold θ, some voted centers will be unconnected. As a result, the LiDAR instances will miss a portion of foreground points. In contrast, camera instances are derived from frustums. A frustum encompasses a large area and may include many noise points. Therefore, the shapes of LiDAR and camera instances are imprecise. Additionally, the shape distributions of the bi-modal instances diverge significantly. These issues pose difficulties in effective feature learning, which is critical for achieving high-quality detection.

To address these issues, we propose a Bi-modal Instance-based Prediction module. This module aligns the shapes of bi-modal instances and then outputs the final prediction results based on the shape-aligned instances. It contains four parts: instance feature extraction, instance feature interaction, instance shape alignment, and the final prediction head.

1) Instance Feature Extraction: The initial step is extracting features from bi-modal instances to facilitate downstream processing tasks. We apply Sparse Instance Recognition (SIR) module [4] in FSD, a PointNet-like module, on instances to

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

extract the instance features. Since LiDAR instances and camera instances have totally different shapes, we use two separate SIR modules to extract the features of LiDAR instances and camera instances. Specifically, we use S to denote the modality, where S can be either L or C, denoting LiDAR or camera. For the S modality, we denote k-th instance as $\mathbf{I}_k^S$, $k = 1, \ldots, m^S$, where m^S represents the total number of instances in modality S. Assume $\mathbf{I}_k^S$ comprises h points. That is, $\mathbf{I}_k^S = \{\mathbf{p}_1^k, \mathbf{p}_2^k, \ldots, \mathbf{p}_h^k\}$. The corresponding point feature is denoted as $\mathcal{G}_k^S = \{\mathbf{g}_1^k, \mathbf{g}_2^k, \ldots, \mathbf{g}_h^k\}$. Then, the feature extraction is formulated as

$$\mathbf{f}_k^S = \texttt{SIR}(\mathcal{G}_k^S), \tag{2}$$

where the instance feature $\mathbf{f}_k^S \in \mathbb{R}^D$ is a single feature vector. D represents the dimension of the feature vector.

2) Instance Feature Interaction: Given the instance features, we propose an Instance Feature Interaction module to leverage the complementarity of LiDAR and camera modalities. This module facilitates interaction among instances by applying self-attention to all instance features. Specifically, given the extracted LiDAR instance features $\mathbf{f}^L = \{\mathbf{f}_1^L, \ldots, \mathbf{f}_{m^L}^L\}$ and camera instance features $\mathbf{f}^C = \{\mathbf{f}_1^C, \ldots, \mathbf{f}_{m^C}^C\}$, we combine them into bi-modal instance features as $\mathbf{f} = \mathbf{f}^L \cup \mathbf{f}^C$. m^L represents the number of LiDAR instances and m^C represents the number of camera instances. Next, we have

$$\mathbf{f}' = \texttt{SelfAttn}(\mathbf{f}), \tag{3}$$

where $\mathbf{f}' = \{\mathbf{f}_1', \ldots, \mathbf{f}_m'\}$ represents the updated instance features. $m = m^L + m^C$ represents the number of instances.

3) Instance Shape Alignment: As mentioned earlier, the bi-modal instances have totally different shape distributions, which is adverse to performing high-quality bi-modal detection. Therefore, we introduce an Instance Shape Alignment module to produce shape-aligned instances. Specifically, we divide $\mathbf{f}'$ into $\mathbf{f}_L'$ and $\mathbf{f}_C'$. Then, we use two separate MLP heads to predict *reference* boxes for bi-modal instances. Then, by placing a predicted reference box into the 3D point cloud space, we include points that fall within this box and exclude those points that are outside of the box. As a result, we obtain the shape-aligned instance of this box.

4) Final Prediction Head: Compared with the initial bi-modal instances, the instances that are aligned in shapes exhibit enhanced precision and more consistent distribution of their shapes. This improvement makes them suitable for the prediction of final bounding boxes. Specifically, another SIR module, self-attention module, and MLP head are utilized to produce the *final* bounding boxes based on these shape-aligned instances. The bounding box is defined by its central coordinates, dimensions, and orientation.

D. Bi-Modal Instance Label Assignment

Assigning an instance to the correct ground truth bounding box is challenging due to these noise points. In this section, we aim to design an effective assignment strategy for bi-modal instances. Before introducing our assignment strategy, we first take a quick overview of the assignment strategy used in LiDAR-only FSD. FSD regards the instance whose center falls into GT boxes as positive, where the center of an instance is defined as the average of all the points in this instance. We name this strategy *point-in-box assignment*. This works for most LiDAR instances.

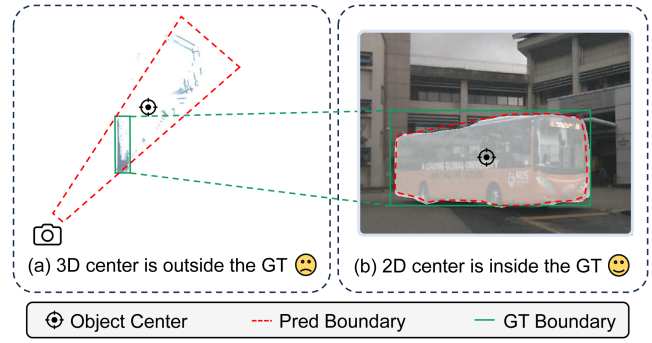

Fig. 4. The motivation of two-stage assignment. The center of a camera instance is hard to fall into 3D GT boxes due to noise points as (a) shows. However, it is easy to assign this camera instance to the corresponding GT on the 2D plane as (b) demonstrates.

However, this strategy could hardly work for camera instances. A camera instance usually has many noise points. The noise points are divided into two types: noise background points and noise foreground points. (i) Camera instances usually contain a large number of irrelevant background points because the frustums are extended along the depth direction and cover a large area as Fig. 4 shows. (ii) The mask generated for an object may inadvertently encompass segments of an adjacent object, especially in cases where two objects are in a direct line with the camera's viewpoint but are located at different distances from the camera. This introduces noise into the camera instance, manifesting as notable depth disparities within foreground points.

These noise points, which are challenging to eliminate, result in displacements of the centers. Such displacements can cause the centers of camera instances to deviate outside their corresponding ground truth bounding boxes. As a result, a substantial number of camera instances are erroneously assigned as negative during the point-in-box assignment process.

To address this issue, we propose a Bi-modal instance assignment as Algorithm 1 shows. Briefly, our assignment strategy is divided into two stages: the first stage is a LIDAR-based assignment stage, which operates in 3D space and is an enhanced version of the point-in-box assignment strategy. The second stage is a camera-based assignment stage, which operates in 2D space and assigns instances based on the 2D similarity between the instances and the projected ground truths.

1) Stage 1. LiDAR-Based Assignment: In FSD, the point-in-box strategy simply averages the coordinates of all points in an instance to obtain the center of this instance. In order to eliminate the influence of irrelevant background points, we introduce a weighted averaging approach. Specifically, the k-th bi-modal instance is denoted as $\mathbf{I}_k = \{\mathbf{p}_1^k, \ldots, \mathbf{p}_l^k, \ldots, \mathbf{p}_h^k\}$. h represents the number of points in $\mathbf{I}_k$ and $l = 1, \ldots, h$ represents the index

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Bi-modal Instance Assignment Strategy.

Input: Predicted 3D Instances: $\mathbf{I}$,
GT bounding boxes $\mathbf{B}^*$.
Output: Assigned labels $\mathbf{L}$.

1: $L_i = NaN$ for all L_i in $\mathbf{L}$ # Initialize to NaN.
2: *Stage-1: LiDAR-based Assignment*
3: **for** $\mathbf{I}_i$ in $\mathbf{I}$ **do**
4: $c_i = \texttt{Center}(\mathbf{I}_i)$. # `Center` is defined in Section IV-D1.
5: **for** $\mathbf{B}^*_j$ in $\mathbf{B}^*$ **do**
6: **if** c_i falls inside $\mathbf{B}^*_j$ **then**
7: Assign $\mathbf{B}^*_j$ to $\mathbf{I}_i$, i.e., $L_i = j$.
8: **break**
9: *Stage-2: Camera-based Assignment*
10: *Define* $\mathcal{O}$, such that $\mathcal{O}(\mathbf{I}_i) = L_i$.
11: $\mathbf{I} \leftarrow \{\mathbf{I}_i | \mathcal{O}(\mathbf{I}_i)\ is\ NaN\}$. # Select instances that are still unassigned after Stage-1.
12: **for** $\mathbf{I}_i$ in $\mathbf{I}$ **do**
13: Initialize α. # Initialize the threshold.
14: **for** $\mathbf{B}^*_j$ in $\mathbf{B}^*$ **do**
15: $\beta = \mathcal{S}(\mathbf{I}_i, \mathbf{B}^*_j)$. # 2D similarity $\mathcal{S}$ is defined in Section IV-D2.
16: **if** $\beta > \alpha$ **then**
17: $\alpha = \beta$. # Update the threshold.
18: $L_i = j$.
19: **return** $\mathbf{L}$

of the point. In Section IV-B1, we produce the foreground classification scores for all the points in the point cloud. Therefore, for $\mathbf{p}^k_l$, we have the corresponding classification score s^k_l. With the help of s^k_l, we define the weighted center of $\mathbf{I}_k$ as

$$\texttt{Center}(\mathbf{I}_k) = \frac{\sum_{l=1}^{h} s^k_l \cdot \mathbf{p}^k_l}{\sum_{l=1}^{h} s^k_l}. \tag{4}$$

Then, we use $\texttt{Center}(\mathbf{I}_k)$ as the center of $\mathbf{I}_k$ to perform point-in-box assignment. We call the whole process above LiADR-based assignment.

2) Stage 2. Camera-Based Assignment: Although (4) refines the centers, the center of a camera instance deviates outside the GT box still happens. There are two reasons: (i) The predicted classification scores may be incorrect. (ii) (4) does not resolve the issues with the notable depth disparities within the foreground points. To rectify this, we implement a second assignment stage named the camera-based assignment stage. To avoid conflicts between the LiDAR-based assignment and the camera-based assignment, the second stage only serves for the camera instances that are still unassigned after the first stage.

The second assignment stage assigns labels by 2D similarity. Specifically, the 2D instance segmentation joint produces 2D masks and 2D bounding boxes of the foreground objects. Therefore, for the j-th remaining unassigned camera instance $\mathbf{I}^C_j$, we have a corresponding 2D bounding box $\mathbf{V}_j$. Besides, we can project the d-th 3D ground-truth box $\mathbf{B}^*_d$ onto 2D image plane to obtain the 2D box $\mathbf{V}^*_d$. As a result, we can compute the 2D similarity $\mathcal{S}$ between $\mathbf{I}^C_j$ and $\mathbf{B}^*_d$ by

$$\mathcal{S}(\mathbf{I}^C_j, \mathbf{B}^*_d) = \mathrm{IoU}_{2D}(\mathbf{V}_j, \mathbf{V}^*_d). \tag{5}$$

where IoU_{2D} means 2D Intersection over Union.

Given $\mathcal{S}(\mathbf{I}^C_j, \mathbf{B}^*_d)$, we then follow the commonly-used "max IoU" [64], [65] assignment strategy to assign labels. For camera instance $\mathbf{I}^C_j$, "max IoU" strategy aims to find the 3D GT box $\mathbf{B}^*_e$, e is the index, where $\mathbf{B}^*_e$ has the max $\mathcal{S}(\mathbf{I}^C_j, \mathbf{B}^*_e)$ value among all the GT boxes. If the maximum value $\mathcal{S}(\mathbf{I}^C_j, \mathbf{B}^*_e)$ is greater than the threshold α, we assign $\mathbf{I}^C_j$ to $\mathbf{B}^*_e$ as lines 11-17 of Algorithm 1 shows.

After these two stages, those instances still not associated with any GTs are assigned as negative. This two-stage assignment strategy is used in both Section IV-C3 for generating reference boxes and in Section IV-C4 for generating final boxes.

E. Loss

Our framework comprises four heads: a head is dedicated to classifying and voting points in Section IV-B1. Two heads are dedicated to generating LiDAR reference bounding boxes and camera reference bounding boxes in Section IV-C3. And one head is used for generating final bounding boxes in Section IV-C4.

The head used for producing bounding boxes is divided into two branches, namely the regression branch and the classification branch. The regression branches regress the 3D bounding box and velocity for each instance. The classification branches are used to classify instances. We use L1 loss for regression branches and Focal Loss [66] for classification branches, respectively.

Formally, the loss of the first head is denoted as $\mathcal{L}_P = \mathcal{L}^{cls}_P + \mathcal{L}^{vote}_P$, where $\mathcal{L}^{cls}_P$ and $\mathcal{L}^{vote}_P$ is the point classification loss and point voting loss in FSD [4]. The loss of LiDAR reference box head is $\mathcal{L}_L = \mathcal{L}^{cls}_L + \mathcal{L}^{reg}_L$, where $\mathcal{L}^{cls}_L$ and $\mathcal{L}^{reg}_L$ is the classification and regression loss for LiDAR instances. The loss of camera reference box head is $\mathcal{L}_C = \mathcal{L}^{cls}_C + \mathcal{L}^{reg}_C$. The loss of final box head is $\mathcal{L}_F = \mathcal{L}^{cls}_F + \mathcal{L}^{reg}_F$.

In total, we have

$$\mathcal{L}_{total} = \mathcal{L}_P + \mathcal{L}_L + \mathcal{L}_C + \mathcal{L}_F, \tag{6}$$

where the loss weight of each item is the same.

V. Experiments

A. Setup

1) Dataset. nuScenes: Our experiment is mainly conducted on the nuScenes dataset [10], which provides a total of 1000 scenes. Each scene is about 20 seconds long and contains collected data from a 32-beam LiDAR along with six cameras. The effective detection area of nuScenes covers a 100 m $\times$ 100 m. The annotations are divided into 10 categories. The official metrics of nuScenes is mean Average Precision (mAP) and nuScenes Detection Score (NDS). The NDS is calculated based on mAP, mATE (translation error), mASE (scale error), mAOE

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

(orientation error), mAVE (velocity error), and mAAE (attribute error).

2) Dataset. Waymo Open Dataset (WOD): To further showcase our approach, we test our method on the widely-used Waymo Open Dataset (WOD) [11]. The WOD has 1150 sequences, broken down into 798 for training, 202 for validation, and 150 for testing. It totally contains more than 200 k frames. WOD is equipped with a 64-beam LiDAR and 5 cameras. However, the cameras do not cover a full 360-degree view. There is a blind area at the back. Objects in the WOD can be detected up to 75 meters away, covering an area sized 150 m $\times$ 150 m. For metrics, WOD employs mAP and mAPH, where mAPH represents mean average precision weighted by heading. In WOD, objects are categorized into LEVEL 1 or LEVEL 2. An object is marked as LEVEL 2 if the number of points in this object is less than or equal to 5. The other objects are marked as LEVEL 1.

3) Dataset. Argoverse 2: To demonstrate our superiority in long-range detection, we proceed to conduct experiments on the Argoverse 2 dataset [12], abbreviated as AV2. The AV2 contains 150 k annotated frames, 5$\times$ larger than nuScenes. AV2 employs two 32-beam LiDARs to form a 64-beam LiDAR, combined with 7 surrounding cameras. The valid detection distance of AV2 is 200 m (covering 400 m $\times$ 400 m area). In addition, AV2 contains 30 classes, exhibiting challenging long-tail distribution. For metrics, in addition to the mean Average Precision (mAP), AV2 provides a Composite Detection Score (CDS) benchmark. CDS is the average score based on mAP, mATE, mASE, and mAOE.

4) Model: Following FSD [4], we adopt a Sparse-UNet [68] as the LiDAR backbone. The voxel size is set to $[0.2\ m, 0.2\ m, 0.2m]$. With respect to 2D instance segmentation, we use HybridTaskCascade (HTC) [63] pre-trained on nuImages [10]. For the SIR module that utilized for instance feature extraction, we use the same setting as FSD [4]. For the thresholds, we set the threshold τ of the foreground classification to 0.1. The threshold θ of CCL is 0.2 and the threshold α of the "max IoU" strategy is 0.3.

5) Training Scheme: For nuScenes, consistent with previous methods [1], [3], we first train the LiDAR-only detector. We pre-train FSD on nuScenes for 20 epochs. Next, initializing from the pre-trained FSD, we train our model for 6 epochs with CBGS [69]. For the augmentation, following FSD [4], we randomly rotate, scale, and translate the point cloud. We also randomly shuffle the order of points. The optimizer is AdamW [70]. We adopt the one-cycle learning rate policy, with a maximum learning rate of 10^{-4} and a weight decay of 10^{-2}. The model is trained on eight NVIDIA RTX 3090 with a batch size of 8. For AV2, we pre-train FSD for 12 epochs and continue training FSF for another 6 epochs. We adopt the same data augmentation as FSD. We set the initial learning rate as 10^{-4} with a weight decay of 0.05. For Waymo Open Datset [11], FSD is pre-trained for 6 epochs, followed by 6 epochs for our model. The data augmentation is the same as FSD. We use the learning rate of 3×10^{-5} with a weight decay of 0.05. Also, we set the weight decay of the batch norm to 0 and set the batch size as 16 following FSD.

B. Comparison With State-of-the-Art Methods

1) Results on nuScenes: As illustrated in Table I, our proposed method, FSF, outperforms all preceding multi-modal 3D object detection techniques on the official nuScenes detection score (NDS) benchmark. It is particularly remarkable that FSF exhibits superior efficacy in comparison to the leading-edge methods such as BEVFusion [1], [2] and DeepInteraction [67]. Moreover, our method achieves SOTA performance on the five detailed metrics: mean Average Translation Error (mATE), mean Average Scaling Error (mASE), mean Average Orientation Error (mAOE), mean Average Velocity Error (mAVE) and mean Average Attribute Error (mAAE), which further demonstrates the superiority of our approach.

2) Results on Waymo Open Dataset (WOD): Table II compares cutting-edge 3D object detection methods on the Waymo validation set using the LEVEL 2 mAPH metric. Our method FSF outperforms others, achieving the SOTA performance. Specifically, we achieve the SOTA performance of 70.4 mAPH in Pedestrians, and 76.5 mAPH in Cyclists. In summary, FSF achieves a leading performance of 72.3 mAPH, highlighting its excellence in multi-modal 3D object detection.

3) Results on Argoverse 2: AV2 has a much larger perception range than nuScenes, making it a suitable testbed to demonstrate our superiority in long-range detection. The results in Table III show that FSF consistently outperforms previous LiDAR-based state-of-the-art methods by large margins. Notably, we observe an 8 to 10 mAP increase in categories with relatively small sizes, such as Traffic Cone, Motorcycle, and Bicycle. Although previous multi-modal methods like BEVFusion [1] and TransFusion [3] closely approach our results on the nuScenes dataset, the computational burden of their dense BEV feature maps proves to be prohibitive for the AV2 dataset. Specifically, the size of detection area of nuScenes is 100 m $\times$ 100 m while the size of AV2 is 400 m $\times$ 400 m. On AV2, the BEV-feature-map-based methods require 16 times as much memory as nuScenes for BEV-map-related operations. As a result, we can not reproduce them on AV2 due to the limitation of GPU memories.

C. Ablation Studies

In this section, we analyze each module within FSF to understand the significance of each component.

1) Bi-Modal Instances Generation: To reveal the pros and cons of the two kinds of instances, we remove each of them from FSF for ablation. To achieve this, we only choose instances from a single modality as the input of the Bi-modal Instance-based Prediction module. There are several intriguing findings in Table IV:

- The model utilizing solely camera instances demonstrates superior performance in detecting objects of relatively small dimensions compared to the model that relies exclusively on LiDAR instances. Notably, the camera-instance-only model exhibits considerable superiority in identifying objects in categories such as Motorcycle, Bicycle, and Traffic Cone. Conversely, for categories encompassing larger-sized objects, like Car, Bus, and Trailer, the model

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON WITH STATE-OF-THE-ART METHODS ON THE NUSCENES DATASET

Method	Modality	*val*		*test*						
		mAP ↑	NDS ↑	mAP ↑	NDS ↑	mATE ↓	mASE ↓	mAOE ↓	mAVE ↓	mAAE ↓
DETR3D [29]	C	34.9	43.4	41.2	47.9	0.641	0.255	0.394	0.845	0.133
BEVDet4D [26]	C	42.1	54.5	45.1	56.9	0.511	0.241	0.386	0.301	0.121
BEVFormer [24]	C	41.6	51.7	48.1	56.9	0.582	0.256	0.375	0.378	0.126
SECOND [35]	L	52.6	63.0	52.8	63.3	-	-	-	-	-
CenterPoint [36]	L	59.6	66.8	60.3	67.3	0.262	0.239	0.361	0.288	0.136
FSD* [4]	L	62.5	68.7	-	-	-	-	-	-	-
VoxelNeXt [5]	L	63.5	68.7	64.5	70.0	0.268	0.238	0.377	0.219	0.127
TransFusion-L [3]	L	64.9	69.9	65.5	70.2	0.256	0.240	0.351	0.278	0.129
LargeKernel3D [38]	L	63.3	69.1	65.3	70.5	0.261	0.236	0.319	0.268	0.133
FUTR3D [52]	C+L	64.5	68.3	-	-	-	-	-	-	-
MVP [59]	C+L	67.1	70.8	66.4	70.5	0.263	0.238	0.321	0.313	0.134
PointAugmenting [9]	C+L	-	-	66.8	71.0	0.254	0.236	0.362	0.266	0.123
UVTR [51]	C+L	65.4	70.2	67.1	71.1	0.306	0.245	0.351	0.225	0.124
TransFusion [3]	C+L	67.5	71.3	68.9	71.6	0.259	0.243	0.359	0.288	0.127
BEVFusion [1]	C+L	67.9	71.0	69.2	71.8	-	-	-	-	-
BEVFusion [2]	C+L	68.5	71.4	70.2	72.9	0.261	0.239	0.329	0.260	0.134
DeepInteraction [67]	C+L	69.9	72.6	**70.8**	73.4	0.257	0.240	0.325	0.245	0.128
FSF (Ours)	C+L	**70.4**	**72.7**	70.6	**74.0**	**0.246**	**0.234**	**0.318**	**0.211**	**0.123**

We do not use test-time augmentation or model ensemble. *: Reimplemented by us. We mark the official benchmark of nuScenes as Gray. C: camera modality. L: LiDAR modality.

TABLE II
PERFORMANCE ON THE WAYMO VALIDATION SET MEASURED BY LEVEL 2 MAPH

Methods	Modality	*Overall*	*Vehicle*	*Pedestrian*	*Cyclist*
PointPillars [32]	L	57.8	63.1	50.3	59.9
LiDAR-RCNN [71]	L	61.3	67.9	51.7	64.4
3D-MAN++† [72]	L	63.0	-	-	-
PVRCNN [37]	L	63.3	68.4	57.6	64.0
TransFusion-L‡ [3]	L	64.9	65.1	63.7	65.9
CenterPoint [36]	L	67.6	68.4	65.8	68.5
FSD* [4]	L	69.7	68.5	68.0	72.5
PointAugmenting [9]	C+L	66.7	62.2	64.6	73.3
TransFusion [3]	C+L	65.5	65.1	64.0	67.4
DeepFusion [57]	C+L	67.0	-	-	
LoGoNet [73]	C+L	71.4	70.7	69.9	73.5
FSF	C+L	**72.3**	70.0	70.4	76.5

All the methods are under the single-frame setting for fair comparison. †: Baseline of DeepFusion, ‡: Baseline of TransFusion *: Baseline of ours.

that only uses LiDAR instances outperforms the camera-instance-only model.

- Combining these two kinds of instances, the performance is improved in all categories. It is noteworthy that the performance is significantly improved in some classes where the LiDAR-instance-only model and camera-instance-only model have similar performance, such as Bus and Pedestrian. This proves the complementarity of these bi-modal instances.

2) Instance Shape Alignment: To verify the effectiveness of our Shape Alignment module, we design a model without shape alignment, which we refer to as FSF-M (FSF-Misaligned). Specifically, we directly use the reference boxes in Fig. 3 as the final results. The last two rows of Table IV show the comparison between FSF-M and FSF , where refining instances give us a 1.8 mAP boost. Without the shape alignment, the performance degrades in all categories consistently, which indicates that shape alignment has an essential impact on the performance.

3) Instance Feature Interaction: In this section, we examine the role of the Instance Feature Interaction module in Section IV-C2. Specifically, since the inputs and outputs of this module are the same (i.e., instance features), we can conduct the ablation study by simply removing this module. As Table V shows, the instance feature interaction module yields a 0.2 mAP and 0.6 NDS improvement. In detail, this module notably decreases orientation error (from 0.345 to 0.317) and velocity error (from 0.244 to 0.218).

4) Two-Stage Assignment: To verify the effectiveness of the two-stage assignment, we make comparisons between our strategy with different assignment strategies: Hungarian assignment [74], Point-in-box assignment [4], LiDAR-based assignment, LiDAR-based assignment with the BEV-distance-based assignment (introduced later) and our proposed two-stage assignment. Specifically, for the Hungarian assignment, we adopt the implementation from DETR3D [29]. The point-in-box strategy corresponds to the assignment used in FSD [4]. The LiDAR-based assignment is the upgraded version of the point-in-box assignment with the help of (4). For the BEV-distance-based assignment, different from the camera-based assignment that uses 2D similarity, this assignment utilizes 3D similarity, defined as the reciprocal of the distance between the center of an instance and the center of a GT box in BEV space. Then, we replace our original two-stage assignment with the alternative approach. These replacements are applied in both the generation of reference boxes in Section IV-C3 and final boxes in Section IV-C4. The results are shown in Table VI. We reach these conclusions.

- The model adopting the Hungarian Assignment fails to converge. The reason is as follows: FSF generates camera instances and LiDAR instances concurrently. So an object may concurrently correspond to a LiDAR instance and a camera instance. We denote these two instances as a LiDAR-camera instance pair. The instances in a pair tightly overlap with each other. Hungarian Assignment chooses one instance in a pair as positive and the other one is regarded as negative. This confuses the classification head

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON WITH STATE-OF-THE-ART METHODS ON ARGOVERSE 2 VALIDATION SPLIT

	Methods	Average	Vehicle	Bus	Pedestrian	Box Truck	C-Barrel	Motorcyclist	MPC-Sign	Motorcycle	Bicycle	A-Bus	School Bus	Truck Cab	C-Cone	V-Trailer	Bollard	Sign	Large Vehicle	Stop Sign	Stroller	Bicyclist
mAP	CenterPoint‡ [36]	13.5	61.0	36.0	33.0	26.0	22.5	16.0	16.0	12.5	9.5	8.5	7.5	8.0	8.0	7.0	25.0	6.5	3.0	28.0	2.0	14
	CenterPoint* [36]	22.0	67.6	38.9	46.5	40.1	32.2	28.6	27.4	33.4	24.5	8.7	25.8	22.6	29.5	22.4	37.4	6.3	3.9	16.9	0.5	20.1
	VoxelNeXt [5]	30.5	72.0	39.7	63.2	39.7	64.5	46.0	34.8	44.9	40.7	21.0	27.0	18.4	44.5	22.2	53.7	15.6	7.3	40.1	11.1	34.9
	FSD [4] (Baseline)	28.2	68.1	40.9	59.0	38.5	42.6	39.7	26.2	49.0	38.6	20.4	30.5	14.8	41.2	26.9	41.8	11.9	5.9	29.0	13.8	33.4
	FSF	**33.2**	70.8	44.1	60.8	40.2	50.9	48.9	28.3	60.9	47.6	22.7	36.1	26.7	51.7	28.1	41.1	12.2	6.8	27.7	25.0	41.6
CDS	CenterPoint* [36]	17.6	57.2	32.0	35.7	31.0	25.6	22.2	19.1	28.2	19.6	6.8	22.5	17.4	22.4	17.2	28.9	4.8	3.0	13.2	0.4	16.7
	VoxelNeXt [5]	23.0	57.7	30.3	45.5	31.6	50.5	33.8	25.1	34.3	30.5	15.5	22.2	13.6	32.5	15.1	38.4	11.8	5.2	30.0	8.9	25.7
	FSD [4] (Baseline)	22.7	57.7	34.2	47.5	31.7	34.4	32.3	18.0	41.4	32.0	15.9	26.1	11.0	30.7	20.5	30.9	9.5	4.4	23.4	11.5	28.0
	FSF	**25.5**	59.6	35.6	48.5	32.1	40.1	35.9	19.1	48.9	37.2	17.2	29.5	19.6	37.3	21.0	29.9	9.2	4.9	21.8	18.5	32.0

C-Barrel: construction barrel. MPC-Sign: mobile pedestrian crossing sign. A-Bus: articulated bus. C-Cone: construction cone. V-Trailer: vehicular trailer. ‡: provided by authors of AV2 dataset. *: reimplemented by FSD. Some categories are excluded from the table due to the limited number of instances they contain. However, the average results consider all categories, even those that are omitted.

TABLE IV
ABLATION STUDY OF THE INSTANCE GENERATION MODULE AND SHAPE ALIGNMENT MODULE

Cam. Instance	LiDAR Instance	Shpae Align.	NDS	mAP	Car	Truck	Bus	Trailer	C.V.	Ped.	Motor.	Bicyc.	T.C.	Barrier
✓			68.8	63.1	80.5	54.3	72.1	34.6	29.0	85.4	72.3	68.8	78.0	64.0
	✓		68.7	62.5	83.9	56.4	73.4	41.4	27.4	84.3	69.5	55.6	72.4	60.7
✓	✓		71.5	68.6	85.2	60.2	75.0	41.5	32.9	87.4	77.2	72.0	80.1	74.1
✓	✓	✓	72.7	70.4	86.1	62.5	76.8	44.8	34.4	88.6	78.7	73.7	82.6	75.5

C.V.: Construction Vehicle. Ped.: Pedestrian. Motor.: Motorcycle. Bicyc.: Bicycle. T.C: Traffic Cone. Gray Cell : By comparing the results generated by the LiDAR-instance-only model and camera-instance-only model, we highlight the results in which one performed better than the other.

TABLE V
ABLATION STUDY OF INSTANCE FEATURE INTERACTION MODULE

Interaction	mAP ↑	NDS ↑	mATE ↓	mASE ↓	mAOE ↓	mAVE ↓	mAAE ↓
×	70.2	72.1	0.279	0.247	0.345	0.244	0.187
✓	70.4	72.7	0.277	0.247	0.317	0.218	0.186

TABLE VI
ABLATION STUDY OF DIFFERENT ASSIGNMENT METHODS

Assignment First Stage	Assignment Second Stage	mAP	NDS
Hungarian	-	Not converged	Not converged
Point-in-box	-	68.7	71.9
LiDAR-based	-	69.0	72.1
LiDAR-based	BEV-dist.-based	69.2	72.2
LiDAR-based	Camera-based	**70.4**	**72.7**

This experiment reveals the superiority of our proposed assignment. BEV-dist.-based: BEV-distance-based.

since these two instances actually represent a single object and make the model fail to converge.

- The model using point-in-box assignment converges but shows an inferior performance to the LiDAR-based assignment due to the disturbance of irrelevant background points.
- Performing a BEV-distance-based assignment in the second stage helps a bit. However, it shows an inferior performance to our proposed two-stage assignment. This is because it regards a large number of camera instances as negative as Section IV-D discussed.

TABLE VII
ABLATION STUDY OF DIFFERENT 2D MODELS

Method	2D Performance Mask*	2D Performance Box*	Mask/Box	3D Performance mAP	3D Performance NDS
Mask RCNN [75]	38.4	47.8	Mask	69.4	72.3
HTC [63]	46.4	57.3	Box	69.8	72.5
			Mask	70.4	72.7

2D performance is evaluated on nuImage while 3D performance is evaluated on nuScenes. Mask*: Mask mAP. Box *: Box mAP. The result reveals that our method is robust against the 2D models.

5) Robustness Against 2D Predictions: It would be a major concern the model is robust with respect to the 2D part. To answer this question, we change the default HTC to a basic Mask RCNN whose 2D Mask AP on nuImages is only 38.4. As Table VII shows, although the 2D performance has a significant drop after degrading the 2D part, the 3D performance of FSF only has 1 mAP drop. This is reasonable. The mission of the 2D part is to detect the objects that are ignored by the LiDAR-only baseline. While the 2D precision significantly decreases, the 2D model maintains a high recall rate, effectively addressing the primary issues associated with a LiDAR-only baseline. Besides, we also conduct an experiment that degrades 2D masks to 2D boxes for producing frustum. The result shows that downgrading the masks to boxes has minimal impact on 3D detection. Representing an object as a mask or box only affects the shape

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
ABLATION STUDY OF USING THE IMAGE FEATURE

Image features	mAP ↑	NDS ↑	mATE ↓	mASE ↓	mAOE ↓	mAVE ↓	mAAE ↓
×	70.4	72.7	0.277	0.247	0.317	0.218	0.186
✓	70.7	72.9	0.274	0.247	0.307	0.223	0.187

Utilizing image features does not result in a significant improvement. As a result, we make it an optional choice.

TABLE IX
DIFFERENT FUSION STRATEGIES FOR SPARSE DETECTION ARCHITECTURE

Method	Modality	mAP	NDS
FSD [4]	L	62.5	68.7
FSD + Score Painting [8]	L+C	66.9	70.9
FSD + Feature Painting [9]	L+C	67.3	71.2
FSD + Virtual Point [59]	L+C	67.6	71.5
FSF	L+C	70.4	72.7

Score Painting [8]: painting semantic scores. Feature Painting [9]: painting the image features. Virtual point [59]: employing virtual points to augment point cloud.

of the camera instance. However, our shape alignment module effectively solves the problem associated with the shapes.

6) The Effect of Image Features: To investigate the effectiveness of image features, we conducted an experiment in the following manner. First, we pre-save the mask result generated by HTC. Then each point is painted with the image feature derived from the image backbone. Then, the image backbone is supervised by the losses of FSF . As illustrated in Table VIII, employing image features yields a minor improvement of 0.3 mAP in our methodology. Training image backbones consumes a significant amount of time and GPU memory. To ensure the efficiency of FSF , we do not use the image feature in our method.

D. Comparison With Alternatives to FSF

In this section, we reproduce several alternatives that can be adapted for fully sparse architecture and conduct a comprehensive comparison between them and FSF.

1) Score Painting: The most straightforward approach is using PointPainting [8], where the 2D semantic scores are painted onto points. The results in Table IX show simple point painting could obtain considerable improvement but is largely inferior to FSF.

2) Feature Painting: Feature painting [9], [57], [58] paints image features to the point cloud. Features contain richer information than semantic scores. We adapt the painting strategy in PointAugmenting [9] on FSD. Compared with score painting, feature painting has an improvement of 0.4 mAP.

3) Virtual Point: MVP [59] utilizes virtual points to augment the original point cloud. Specifically, MVP utilizes 2D instance segmentation to generate 2D masks. Next, MVP uniformly samples 2D points in a 2D mask. Then, it projects the LiDAR points onto the 2D image plane. Each sampled 2D point finds the nearest projected LiDAR point and uses the depth of this LiDAR point as its own depth. Finally, given the depth, the sampled 2D point is unprojected to 3D space and augments the original point cloud. Then the augmented point clouds are sent into a LiDAR-based detector. For a fair comparison, we use the official code to generate virtual points and then feed the augmented point cloud into FSD. Table IX demonstrates that adopting MVP achieves better performance than the painting method above, but is still worse than ours.

TABLE X
PERFORMANCE OF DIFFERENT RANGES ON ARGOVERSE 2

	Overall mAP	0m-50m Avg.	0m-50m Motor.	0m-50m Bicyc.	0m-50m C.B.	50m-100m Avg.	50m-100m Motor.	50m-100m Bicyc.	50m-100m C.B.
FSD	28.1	41.6	57.3	57.4	66.1	10.9	8.8	17.0	13.8
FSF	33.2	45.3	57.9	65.1	73.7	17.2	36.4	31.1	25.2

FSF significantly improves the performance on faraway objects relative to the FSD [4]. Avg.: the average mAP of all classes. Motor.: Motorcyclist. Bicyc.: Bicyclist. C.B.: Construction Barrel. We select three representative categories of small objects for demonstration.

4) Discussion. Why is FSF Superior to the Alternatives?: Although all three alternatives achieve improvement on the LiDAR-only baseline, they are significantly worse than FSF. We owe the superior performance of FSF to two aspects:

- Semantic scores and image features are rich in semantic-level information but lack instance-level information. However, instance-level information is vital to detection since it greatly helps recognize objects within the same category.
- The virtual point approach employs 2D instance segmentation. However, it only augments the point cloud. The objects with few points are still likely to be missed out. Nevertheless, FSF ensures that a 2D mask will produce a camera instance even when an object contains only one point. This is greatly helpful in detecting objects with a minimal number of points.

E. Long Range Detection

FSF achieves state-of-the-art performance on the Argoverse 2 dataset, which has a very large perception area ($400\ \text{m} \times 400\ \text{m}$). We conduct the following experiments to showcase the superiority of FSF in long-range detection.

1) Range-Conditioned Performance: The image is dense and of high resolution, making it helpful for detecting distant objects. We conduct the range-conditioned performance to validate the effectiveness of our method. Table X demonstrates the performance conditioned on different perception ranges. As Table X shows, FSF greatly enhances the capabilities of FSD in long-range scenarios. Particularly, detecting small objects at a long distance is especially challenging, which is a good testbed to showcase the effectiveness of FSF. Therefore, we select three small object categories that are frequently encountered: Motorcyclist, Bicyclist as representatives. In the range spanning from 50 m to 100 m, we observe a remarkable improvement in mAP for these small objects.

2) Latency and Memory Footprint Analysis: Here we emphasize the superiority of FSF in long-range detection by evaluating the inference latency and memory footprint. The latency is tested on RTX 3090 with batch size 1. Our code is based on MMDetection3D [76] and the IO time is excluded. For comparison, we use the officially released codebases of CenterPoint and TransFusion to obtain their results.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
INFERENCE LATENCY AND MEMORY FOOTPRINT COMPARISON ON THE ARGOVERSE 2 DATASET, COVERING A 400 M × 400 M AREA

Method	Modality	Latency (ms)				Memory (GB)
		Image	LiDAR	Prediction	Overall	
TransFusion-L	L	-	198	122	320	14.5
CenterPoint	L	-	190	42	232	8.1
FSD	L	-	56	41	**97**	**3.8**
TransFusion	C+L	30	198	156	384	17.3
FSF (ours)	C+L	41	56	44	**141**	**6.9**

First, we introduce the specific settings in Table XI:

- *Image Process:* In TransFusion, the image component is constructed on a ResNet50 backbone and operates on images of size 800×320. FSF adopts the same backbone and image size as TransFusion but extends the image backbone with a 2D head for generating camera instances.
- *LiDAR Process:* CenterPoint, TransFusion-L, and TransFusion employ a VoxelNet-like [33] LiDAR backbone followed by 2D convolutions to generate BEV feature maps. FSD and FSF use the SparseUNet [68] backbone to extract voxel features. Then, unlike BEV-based methods, FSD and FSF construct the point features from voxel features. These point features are used to produce LiDAR instances.
- *Prediction Process:* We define the prediction process as the process of generating 3D bounding boxes from a BEV feature map or 3D instances. The prediction process of each method is as follows:
  (a) CenterPoint uses an MLP head to produce 3D boxes from the features of BEV grids.
  (b) TransFusion operates in three steps: (i) It first utilizes an MLP head to classify the features of BEV grids and then selects top-k grids to form k queries. (ii) TransFusion then utilizes a transformer decoder with global cross-attention to update the feature of queries. The global attention is performed between queries and the feature maps from both image and LiDAR modalities. (iii) It finally outputs 3D bounding boxes based on the features of queries using an MLP head.
  (c) TransFusion-L is identical to TransFusion but omits the step of performing cross-attention between queries and image feature maps.
  (d) FSD and FSF first extract instance features from instances using the SIR module. Then they employ these instance features to generate reference boxes for shape alignment. Finally, FSD and FSF generate 3D bounding boxes from the shape-aligned instances.

As Table XI shows, our FSF method shows a remarkable advantage in both latency and memory footprint over other state-of-the-art multi-modal methods. It is mainly attributed to two reasons:

- FSF does not construct a BEV feature map. Considering a detection area of $400\ \text{m} \times 400\ \text{m}$, creating a BEV feature map over such an extended range would require 16 times more memory compared to a $100\ \text{m} \times 100\ \text{m}$ area. Moreover, conducting 2D convolution operations on this BEV feature map would be computationally intensive and time-consuming.
- FSF has a much faster prediction process than TransFusion. Transfusion needs to perform global attention with the BEV feature map and image feature maps. Meanwhile, FSF directly generates bounding boxes from 3D instances, bypassing the need to process features of the background.

Despite the need to process image modality, our approach remains faster and requires less memory than well-known LiDAR-only methods such as CenterPoint [36].

3) Latency Breakdown: We have provided a more detailed runtime analysis of each component in Table XII. Despite a somewhat long pipeline of FSF, each component is designed to be efficient, ensuring that the overall processing speed remains rapid.

TABLE XII
BREAKDOWN OF FSF LATENCY ON THE ARGOVERSE2 DATASET USING THE RTX 3090 GPU

Module Name	Latency (ms)
Camera Instance Generation	**41.1**
2D Instance Segmentation	39.8
Point Cropping	1.3
LiDAR Instance Generation	**56.2**
LiDAR Backbone	42.3
Point Classification + Center Voting + Connecting	7.8
Grouping by CCL	6.1
Bi-modal Instance-based Prediction	**43.9**
Instance Feature Extraction 1	9.2
Self-Attn 1	5.8
Head 1	1.8
Point Cropping	2.2
Feature Extraction 2	9.2
Self-Attn 2	5.8
Head 2	1.8
NMS	8.1

The order of the modules in the table corresponds to their arrangement in Fig. 3.

F. Evaluating FSF and FSD in Hard Scenarios

In Fig. 2, we hypothesize that multi-modal FSF can enhance the performance of single-modal FSD in two scenarios. The first is when points are very few. The second is when numerous objects are closely packed together. To validate our claims, we conducted the following experiments.

1) Superior Performance for Sparsely-Pointed Objects: We first demonstrate the performance improvement of FSF over FSD under the setting of different numbers of points within objects. We categorize objects based on the number of points they contain, splitting them into four groups: $(0, 10]$, $(10, 30]$, $(30, 100]$, and $(100, \infty)$. Since mAP is calculated at the scene level and each scene contains objects with varying numbers of points, directly evaluating mAP for objects with a specific number of points becomes problematic. Therefore, we adopt *mean average recall* as the metric of Table XIII. Across all categories, FSF consistently outperforms FSD. Notably, in the $(0, 10]$ group, the improvement is most pronounced, showing an overall increase of 4.3 recall. For small objects like bicycles and

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

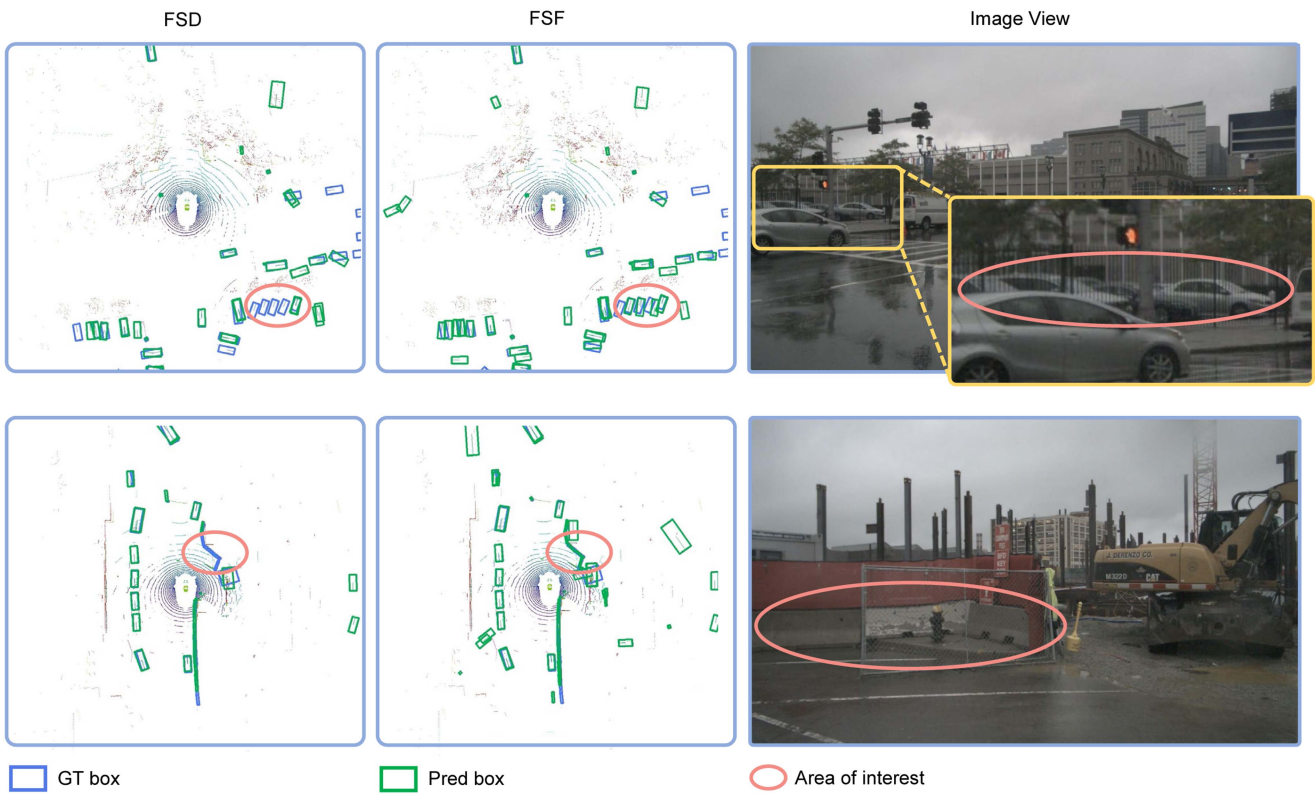

Fig. 5. Qualitative comparison between FSD [4] and FSF. In these scenes, some objects are partially occluded by wire netting, leading to very few LiDAR points. We mark the areas of interest with red circles. With the help of image information, FSF detects more objects than FSD.

TABLE XIII
COMPARISON OF *RECALL* VALUES UNDER THE DIFFERENT NUMBERS OF POINTS WITHIN OBJECTS

Method	#Points	#GT boxes	mAR	Motor.	Bicyc.	T.C.
FSD	0-10	59457	52.3	61.3	69.4	82.1
FSF			56.6 ↑ 4.3	67.7	79.6↑ 10.2	91.7↑ 9.6
FSD	10-30	25735	69.6	93.1	88.2	96.1
FSF			71.9 ↑ 2.3	96.9	92.8	98.9
FSD	30-100	18865	78.1	97.1	94.2	98.1
FSF			80.5 ↑ 2.4	98.9	98.8	98.9
FSD	100-inf	17804	85.2	98.1	94.2	97.1
FSF			87.3 ↑ 1.9	100.0	100.0	98.9

The recall threshold is 0.5 m. Motor.: Motorcyclist. Bicyc.: Bicyclist. T.C.: Traffic Cone. We select three representative categories of small objects for demonstration.

TABLE XIV
COMPARISON FOR FSD AND FSF ACROSS CROWDED SCENES AND ALL SCENES

Method	Scene Type	NDS	mAP	Motor.	Bicyc.	T.C.
FSD	crowded	65.2	55.7	66.0	44.9	55.0
FSF	crowded	70.4	66.8 ↑ 11.1	77.6	69.5	72.6
FSD	all	68.7	62.5	69.5	55.6	72.5
FSF	all	72.7	70.4 ↑ 7.9	78.7	73.7	82.6

Motor.: Motorcyclist. Bicyc.: Bicyclist. T.C.: Traffic Cone. We select three representative categories of small objects for demonstration.

traffic cones, which contain very few points, pure 3D segmentation struggles with recognition. However, with the assistance of 2D segmentation, identification is simplified (10 recall boost).

2) Superior Performance in Crowded Scenarios: This experimental section evaluates the FSD and FSF performance in crowded scenarios. In crowded scenarios, 3D segmentation is prone to group multiple objects as a single instance, whereas 2D segmentation is skilled at differentiating between these objects.

To better understand the challenges posed by crowded scenarios, we define two terminologies: *crowded box* and *crowded scene*. Considering all the boxes in the BEV perspective, a GT BEV box B_i^* is designated as a crowded box if its 2D center lies within a distance threshold ε_i from the 2D center of another GT box B_j^*. The threshold ε_i is defined as the diagonal length of B_i^*. Thus, if the center-to-center distance between B_i^* and B_j^* falls below ε_i, B_i^* qualifies as a *crowded box*. A *crowded scene* is identified when over 50% of the GT boxes in this scene are *crowded boxes*.

In Table XIV, it is evident that the FSD method experiences a notable performance drop in crowded scenes compared to overall scenarios, with a mAP reduction of 6.8. This decline is primarily attributed to the limitations of 3D instance segmentation in accurately distinguishing objects that are closely packed together. In contrast, our proposed FSF method, which employs 2D instance segmentation, has enhanced capability to identify these objects separately. Specifically, FSF achieves a lower mAP drop of 3.6. FSF has an 11.1 mAP improvement over FSD in crowded scenes, a more significant gain compared to the 7.9 mAP increase observed in overall scenarios.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

G. Qualitative Results

To gain insight into the inner workings of FSF, we present the qualitative results. An interesting case is demonstrated in Fig. 5. Here, FSD [4] fails to detect objects that are partially occluded by wire netting. Due to the wire netting, a significant portion of the lasers emitted by LiDAR is intercepted, resulting in few point clouds on these objects. It is difficult for LiDAR-only methods to identify these objects. However, using images, the masks of these objects are easily identified, allowing FSF to detect them successfully.

VI. Conclusion

We present the Fully Sparse Fusion (FSF) framework, the first multimodal architecture designed for full sparsity. FSF is an instance-level fusion method that contains no dense BEV feature maps. We introduce a Bi-modal Instance Generation module that performs joint 2D and 3D instance segmentation to produce bi-modal instances. Subsequently, these instances become shape-aligned and are used for bounding box prediction through our Bi-modal Instance-based Prediction Module. Moreover, an effective Bi-modal Assignment Strategy is proposed to properly assign these bi-modal instances. Our FSF framework sets new state-of-the-art performance on three large-scale datasets: nuScenes, WOD, and Argoverse 2, covering both short-range and long-range scenarios. Notably, FSF exhibits exceptional capability and efficiency in long-range detection, significantly reducing latency and memory overhead.

References

[1] T. Liang et al., "BEVFusion: A simple and robust LiDAR-camera fusion framework," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 10 421–10 434.

[2] Z. Liu et al., "BEVFusion: Multi-task multi-sensor fusion with unified bird's-eye view representation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 2774–2781.

[3] X. Bai et al., "TransFusion: Robust LiDAR-camera fusion for 3D object detection with transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 1090–1099.

[4] L. Fan, F. Wang, N. Wang, and Z.-X. Zhang, "Fully sparse 3D object detection," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 351–363.

[5] Y. Chen, J. Liu, X. Zhang, X. Qi, and J. Jia, "VoxelNeXt: Fully sparse VoxelNet for 3D object detection and tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 21 674–21 683.

[6] Z. Liu, X. Yang, H. Tang, S. Yang, and S. Han, "FlatFormer: Flattened window attention for efficient point cloud transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 1200–1211.

[7] L. Fan, Y. Yang, F. Wang, N. Wang, and Z. Zhang, "Super sparse 3D object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 10, pp. 12490–12505, Oct. 2023.

[8] S. Vora, A. H. Lang, B. Helou, and O. Beijbom, "PointPainting: Sequential fusion for 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 4604–4612.

[9] C. Wang, C. Ma, M. Zhu, and X. Yang, "PointAugmenting: Cross-modal augmentation for 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 11 794–11 803.

[10] H. Caesar et al., "nuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11 621–11 631.

[11] P. Sun et al., "Scalability in perception for autonomous driving: Waymo open dataset," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 2446–2454.

[12] B. Wilson et al., "Argoverse 2: Next generation datasets for self-driving perception and forecasting," 2023, *arXiv:2301.00493*.

[13] G. Brazil and X. Liu, "M3D-RPN: Monocular 3D region proposal network for object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 9287–9296.

[14] Z. Liu, D. Zhou, F. Lu, J. Fang, and L. Zhang, "AutoShape: Real-time shape-aware monocular 3D object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 15 641–15 650.

[15] Y. Zhang, J. Lu, and J. Zhou, "Objects are different: Flexible monocular 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 3289–3298.

[16] Y. Lu et al., "Geometry uncertainty projection network for monocular 3D object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 3111–3121.

[17] Y. Li, Y. Chen, J. He, and Z. Zhang, "Densely constrained depth estimator for monocular 3D object detection," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2022, pp. 718–734.

[18] Z. Chong et al., "MonoDistill: Learning spatial features for monocular 3D object detection," 2022, *arXiv:2201.10830*.

[19] J. He, Y. Chen, N. Wang, and Z. Zhang, "3D video object detection with learnable object-centric global optimization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 5106–5115.

[20] P. Li, X. Chen, and S. Shen, "Stereo R-CNN based 3D object detection for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 7644–7652.

[21] X. Guo, S. Shi, X. Wang, and H. Li, "LIGA-stereo: Learning LiDAR geometry aware representations for stereo-based 3D detector," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 3153–3163.

[22] Y. Wang, Y. Chen, and Z. Zhang, "FrustumFormer: Adaptive instance-aware resampling for multi-view 3D detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 5096–5105.

[23] C. Reading, A. Harakeh, J. Chae, and S. L. Waslander, "Categorical depth distribution network for monocular 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 8555–8564.

[24] Z. Li et al., "BEVFormer: Learning bird's-eye-view representation from multi-camera images via spatiotemporal transformers," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2022, pp. 1–18.

[25] J. Philion and S. Fidler, "Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3D," in *Proc. 16th Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 194–210.

[26] J. Huang and G. Huang, "BEVDet4D: Exploit temporal cues in multi-camera 3D object detection," 2022, *arXiv:2203.17054*.

[27] Y. Li et al., "BEVDepth: Acquisition of reliable depth for multi-view 3D object detection," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 1477–1485.

[28] C. Yang et al., "BEVFormer v2: Adapting modern image backbones to bird's-eye-view recognition via perspective supervision," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 17 830–17 839.

[29] Y. Wang, V. C. Guizilini, T. Zhang, Y. Wang, H. Zhao, and J. Solomon, "DETR3D: 3D object detection from multi-view images via 3D-to-2D queries," in *Proc. Conf. Robot Learn.*, 2022, pp. 180–191.

[30] Y. Liu, T. Wang, X. Zhang, and J. Sun, "PETR: Position embedding transformation for multi-view 3D object detection," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2022, pp. 531–548.

[31] Z. Wang, Z. Huang, J. Fu, N. Wang, and S. Liu, "Object as query: Lifting any 2D object detector to 3D detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 3768–3777.

[32] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "PointPillars: Fast encoders for object detection from point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 12 697–12 705.

[33] Y. Zhou and O. Tuzel, "VoxelNet: End-to-end learning for point cloud based 3D object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4490–4499.

[34] B. Li, "3D fully convolutional network for vehicle detection in point cloud," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2017, pp. 1513–1518.

[35] Y. Yan, Y. Mao, and B. Li, "SECOND: Sparsely embedded convolutional detection," *Sensors*, vol. 18, no. 10, 2018, Art. no. 3337.

[36] T. Yin, X. Zhou, and P. Krahenbuhl, "Center-based 3D object detection and tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 11 784–11 793.

[37] S. Shi et al., "PV-RCNN: Point-voxel feature set abstraction for 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 10 529–10 538.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

[38] Y. Chen, J. Liu, X. Zhang, X. Qi, and J. Jia, "LargeKernel3D: Scaling up kernels in 3D sparse CNNs," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 13 488–13 498.

[39] Y. Chen, Y. Li, X. Zhang, J. Sun, and J. Jia, "Focal sparse convolutional networks for 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 5428–5437.

[40] H. Zhou et al., "Cylinder3D: An effective 3D framework for driving-scene LiDAR semantic segmentation," 2020, *arXiv:2008.01550*.

[41] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 652–660.

[42] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5105–5114.

[43] Z. Yang, Y. Sun, S. Liu, and J. Jia, "3DSSD: Point-based 3D single stage object detector," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11 040–11 048.

[44] C. R. Qi, O. Litany, K. He, and L. J. Guibas, "Deep hough voting for 3D object detection in point clouds," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 9277–9286.

[45] S. Shi, X. Wang, and H. Li, "PointRCNN: 3D object proposal generation and detection from point cloud," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 770–779.

[46] L. Fan, X. Xiong, F. Wang, N. Wang, and Z. Zhang, "RangeDet: In defense of range view for LiDAR-based 3D object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 2918–2927.

[47] Z. Tian, X. Chu, X. Wang, X. Wei, and C. Shen, "Fully convolutional one-stage 3D object detection on LiDAR range images," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 34 899–34 911.

[48] J. H. Yoo, Y. Kim, J. Kim, and J. W. Choi, "3D-CVF: Generating joint camera and LiDAR features using cross-view spatial feature fusion for 3D object detection," in *Proc. 16th Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 720–736.

[49] J. Ku, M. Mozifian, J. Lee, A. Harakeh, and S. L. Waslander, "Joint 3D proposal generation and object detection from view aggregation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 1–8.

[50] X. Chen, H. Ma, J. Wan, B. Li, and T. Xia, "Multi-view 3D object detection network for autonomous driving," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1907–1915.

[51] Y. Li, Y. Chen, X. Qi, Z. Li, J. Sun, and J. Jia, "Unifying voxel-based representation with transformer for 3D object detection," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 18 442–18 455.

[52] X. Chen, T. Zhang, Y. Wang, Y. Wang, and H. Zhao, "FUTR3D: A unified sensor fusion framework for 3D detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 172–181.

[53] Y. Xie et al., "SparseFusion: Fusing multi-modal sparse representations for multi-sensor 3D object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 17 591–17 602.

[54] T. Huang, Z. Liu, X. Chen, and X. Bai, "EPNet: Enhancing point features with image semantics for 3D object detection," in *Proc. 16th Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 35–52.

[55] S. Xu, D. Zhou, J. Fang, J. Yin, Z. Bin, and L. Zhang, "FusionPainting: Multimodal fusion with adaptive attention for 3D object detection," in *Proc. IEEE Int. Intell. Transp. Syst. Conf.*, 2021, pp. 3047–3054.

[56] S. Pang, D. Morris, and H. Radha, "CLOCs: Camera-LiDAR object candidates fusion for 3D object detection," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 10 386–10 393.

[57] Y. Li et al., "DeepFusion: LiDAR-camera deep fusion for multi-modal 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 17 182–17 191.

[58] Z. Chen et al., "AutoAlign: Pixel-instance feature aggregation for multi-modal 3D object detection," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, 2022, pp. 827–833.

[59] T. Yin, X. Zhou, and P. Krähenbühl, "Multimodal virtual point 3D detection," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 16 494–16 507.

[60] C. R. Qi, W. Liu, C. Wu, H. Su, and L. J. Guibas, "Frustum pointnets for 3D object detection from RGB-D data," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 918–927.

[61] Z. Wang and K. Jia, "Frustum ConvNet: Sliding frustums to aggregate local point-wise features for amodal 3D object detection," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 1742–1749.

[62] L. Di Stefano and A. Bulgarelli, "A simple and efficient connected components labeling algorithm," in *Proc. 10th Int. Conf. Image Anal. Process.*, 1999, pp. 322–327.

[63] K. Chen et al., "Hybrid task cascade for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4974–4983.

[64] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.

[65] W. Liu et al., "SSD: Single shot multibox detector," in *Proc. 14th Eur. Conf. Comput. Vis.*, Springer, 2016, pp. 21–37.

[66] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2980–2988.

[67] Z. Yang, J. Chen, Z. Miao, W. Li, X. Zhu, and L. Zhang, "Deepinteraction: 3D object detection via modality interaction," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 1992–2005.

[68] S. Shi, Z. Wang, J. Shi, X. Wang, and H. Li, "From points to parts: 3D object detection from point cloud with part-aware and part-aggregation network," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 8, pp. 2647–2664, Aug. 2021.

[69] B. Zhu, Z. Jiang, X. Zhou, Z. Li, and G. Yu, "Class-balanced grouping and sampling for point cloud 3D object detection," 2019, *arXiv: 1908.09492*.

[70] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, *arXiv: 1711.05101*.

[71] Z. Li, F. Wang, and N. Wang, "LiDAR R-CNN: An efficient and universal 3D object detector," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 7546–7555.

[72] Z. Yang, Y. Zhou, Z. Chen, and J. Ngiam, "3D-Man: 3D multi-frame attention network for object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 1863–1872.

[73] X. Li et al., "LoGoNet: Towards accurate 3D object detection with local-to-global cross-modal fusion," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 17 524–17 534.

[74] H. W. Kuhn, "The Hungarian method for the assignment problem," *Nav. Res. Logistics Quart.*, vol. 2, no. 1/2, pp. 83–97, 1955.

[75] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2961–2969.

[76] M. Contributors, "MMDetection3D: OpenMMLab next-generation platform for general 3D object detection," 2020. [Online]. Available: https://github.com/open-mmlab/mmdetection3d

Yingyan Li received the BS degree from the University of Chinese Academy of Sciences, Beijing, in 2021. He is currently working toward the PhD degree with the National Laboratory of Pattern Recognition (NLPR), Institute of Automation, Chinese Academy of Sciences (CASIA), Beijing, China, supervised by Prof. Zhaoxiang Zhang. He is also currently with the School of Future Technology, University of Chinese Academy of Sciences (UCAS). His major research interests include 3D perception and autonomous driving.

Lue Fan received the BS degree from Xi'an Jiaotong University, in 2019, and majored in automation. He is currently working toward the PhD degree with the National Laboratory of Pattern Recognition (NLPR), Institute of Automation, Chinese Academy of Sciences (CASIA), Beijing, China, supervised by Prof. Zhaoxiang Zhang. He is also currently with the School of Future Technology, University of Chinese Academy of Sciences (UCAS). His major research interests lie in 3D/2D perception algorithms and applications.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.

Yang Liu received the BS degree from Yingcai Honor College, University of Electronic Science and Technology of China, in 2018. He is currently working toward the PhD degree with the Institute of Automation, Chinese Academy of Sciences. His research interests include 3D computer vision and reconstruction.

Zehao Huang received the BS degree in automatic control from Beihang University, Beijing, China, in 2015. He is currently an algorithm engineer with TuSimple. His research interests include computer vision and image processing.

Yuntao Chen received the bachelor's degree from the University of Science and Technology, Bejing, China, in 2016, and the PhD degree from the Center for Research on Intelligent Perception and Computing (CRIPAC), National Laboratory of Pattern Recognition (NLPR), Institute of Automation, Chinese Academy of Sciences (CASIA), Beijing, in 2021. His research interests include object recognition and 3D scene understanding.

Naiyan Wang received the BS degree from Zhejiang University, in 2011 under the supervision of Prof. Zhihua Zhang, and the PhD degree from the CSE Department, Hong Kong University of Science and Technology, in 2015. He is currently the chief scientist of TuSimple. He leads the Algorithm Research Group in the Beijing branch. His supervisor is Prof. Dit-Yan Yeung. His research interest focuses on applying the statistical computational model to real problems in computer vision and data mining. Currently, he mainly works on the vision-based perception and localization part of autonomous driving. Especially, he integrates and improves the cuttingedge technologies in academia, and makes them work properly in the autonomous truck.

Zhaoxiang Zhang received the bachelor's degree in circuits and systems from the University of Science and Technology of China (USTC), in 2004, and the PhD degree from the National Laboratory of Pattern Recognition (NLPR), Institute of Automation, Chinese Academy of Sciences (CASIA), Beijing, China, in 2009. In 2009, he joined the School of Computer Science and Engineering, Beihang University, and worked as an assistant professor from 2009 to 2011, an associate professor from 2012 to 2015, and the vice-director with the Department of Computer Application Technology from 2014 to 2015. In 2015, he returned to the CASIA, to join as a professor, where he is currently a professor with the Center for Research on Intelligent Perception and Computing. He has published more than 200 papers in reputable conferences and journals. His major research interests include pattern recognition, computer vision, machine learning, and bio-inspired visual computing. He has won the best paper awards in several conferences.

Authorized licensed use limited to: University of Chinese Academy of SciencesCAS. Downloaded on October 18,2024 at 08:29:28 UTC from IEEE Xplore. Restrictions apply.